Deep Reinforcement Learning-Based Hierarchical Motion Planning Strategy for Multirotors

Hean Hua, Yaonan Wang, Hang Zhong, *Member, IEEE*, Hui Zhang, *Member, IEEE*, and Yongchun Fang, *Senior Member, IEEE*

***Abstract*—This article proposes a novel hierarchical motion planning strategy for multirotors, where the virtual goal (VG) oriented deep reinforcement learning (RL) and motion optimization are designed cooperatively to achieve efficient, flexible and smooth navigation in unknown environments. Specifically, the intelligent hierarchical motion planning is achieved in a three-step design. First, the dynamic VG generation algorithm is proposed considering the perception range of onboard sensors and current velocity, which transforms the global navigation into a real-time point-to-VG planning, thereby guaranteeing efficient computation even in resource-limited multirotors. Second, instead of generating motion actions, the upper-layer deep RL is designed to make spatial-temporal decisions of VG online, which outputs time allocation and spatial distribution commands according to current observation. Third, based on upper-layer's decisions, local optimization and control are implemented accordingly. Different from existing solutions, high-performance planning is guaranteed by the online VG oriented intelligent decision making, where the data-driven learning and model-driven optimization are integrated to navigate the multirotors. Comparative experiments are carried out in both physical simulation and indoor environments, which demonstrate the satisfactory performance of the proposed motion planning strategy in terms of feasibility, efficiency, navigation smoothness, and flexibility.**

***Index Terms*—Deep reinforcement learning (RL), hierarchical multirotor planning, local trajectory generation, real-time navigation, real-world experiments.**

Received 23 June 2024; revised 9 November 2024; accepted 11 December 2024. Date of publication 18 March 2025; date of current version 23 May 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2021ZD0114503, and in part by the National Natural Science Foundation of China under Grant 62403190, Grant 62427813, and Grant 62433010. Paper no. TII-24-3124. *(Corresponding author: Yaonan Wang.)*

Hean Hua, Hang Zhong, and Hui Zhang are with the School of Robotics, Hunan University, Changsha 410082, China, and also with the National Engineering Research Center for Robot Visual Perception and Control Technology, Changsha 410082, China (e-mail: huahean@hnu.edu.cn; zhonghang@hnu.edu.cn; zhanghui1983@hnu.edu.cn).

Yaonan Wang is with the College of Electrical and Information Engineering, Hunan University, Changsha 410082, China, and also with the National Engineering Research Center for Robot Visual Perception and Control Technology, Changsha 410082, China (e-mail: yaonan@hnu.edu.cn).

Yongchun Fang is with the Institute of Robotics and Automatic Information System, College of Artificial Intelligence, Nankai University, Tianjin 300353, China, and also with the Tianjin Key Laboratory of Intelligent Robotics, Nankai University, Tianjin 300353, China (e-mail: fangyc@nankai.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TII.2024.3523594, provided by the authors.

Digital Object Identifier 10.1109/TII.2024.3523594

I. Introduction

OVER the past decades, multirotors attract great attention from military and civil applications [1], [2], [3], and meaningful studies have been conducted on the motion planning problem of multirotors [4], [5], [6]. Currently, multirotor is increasingly deployed in complicated environments, such as surveillance, rescuing, and mapping. To achieve satisfactory performance in these applications, it is becoming an urgent demand to develop high-performance motion planning strategies, guaranteeing safe, efficient, and flexible navigation [7]. However, it is difficult to design effective motion planning algorithm for multirotors, especially considering the lack of calculation resources, comprehensive perception [8], and accurate model-reality matches [9], and moreover, the complicated underactuated dynamics adds additional challenges in designing dynamically feasible and flexible trajectories. To address the multirotor motion planning problem, remarkable solutions are proposed, where learning and optimization frameworks are widely adopted in recent years [10], [11].

On one hand, the motion planning problem is transformed into optimization formulation, utilizing the differential flatness property of multirotors [12]. Then, optimization-based methods are proposed to solve the planning problem, where the multirotor navigation is achieved by executing the obtained results [13]. Specifically, by exploring the differential flatness property of multirotor, the complicated underactuated dynamics is projected into the flat output space, where the state-constrained problems are transformed into output-constrained planning, so as to avoid the direct processing of high-dimensional coupling. For example, by selecting proper flat outputs, the states and inputs of multirotors are written as algebraic functions in the flat space, guaranteeing that the optimization-based minimum snap trajectory generation algorithm runs efficiently [14]. Aiming at improving the spatial and temporal performance of the multirotor planning, the waypoint-based trajectory planning is carried out by alternately optimizing the spatial and temporal aspects [15]. Besides, instead of directly solving the quadratic programming optimization problem, a closed-form solution is proposed to calculate polynomial trajectory in [16]. To

1941-0050 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on December 02,2025 at 07:17:45 UTC from IEEE Xplore. Restrictions apply.

accomplish multirotor payload transportation task, the trajectory planning is transformed into nonlinear optimization, where the augmented Lagrangian method is adopted to address the problem [17]. Also, the risk-aware motion planning is focused on in [18], where the linearly constrained mixed integer convex program formulation is utilized to achieve safe motion planning and control for multirotors. The key idea of these proposals is to use the dynamic model knowledge to construct state constraints and further formulate solvable optimization problems. In this way, the minimum-snap, risk-aware, optimal-energy, or other performance metrics can be achieved by employing existing solvers. However, the important real-time performance, spatial-temporal optimality, and computation efficiency requirements cannot be satisfied simultaneously in these proposals. Specifically, the spatial and temporal constraints are usually coupled in the formulation, making it difficult or even impossible to obtain spatial-temporal optimal solutions in real time. To address the problem, heuristic and iterative approaches are adopted to improve the spatial-temporal optimality of current solutions. But these approaches suffer from large computation burden and local-minima problems, which prevents the wide application of multirotor in unknown environments, especially considering that the onboard calculation requires low computation burden and real-time response.

On the other hand, learning-based motion planning methods calculate real-time solutions based on huge amount of experiences, where the nonconvex problem can be handled in a data-driven manner [19], [20]. Different from model-based optimization, learning-based methods generate motion planning results utilizing the generalization ability of deep neural networks. Specifically, an objective idea is to imitate existing demonstrations, which reduces the complicated computation process, and simultaneously outputs imitation results utilizing the generalization ability of neural networks [21]. For example, the time-consuming gradient descent algorithm is adopted to train a supervised neural network, where the learned network achieves two orders of magnitude reduction in computational time for quadrotor trajectory generation [22]. In order to obtain a safe navigation policy, a pretraining algorithm is designed by imitating existing methods in [23]. By designing an imitation learning algorithm, the global optimal trajectories are adopted for training, where the obtained results perform better than human drone pilots in drone racing [24]. The above proposals output real-time decisions by imitating demonstrations, but the outcome suffers from local minima, and its performance depends largely on the quality of demonstrations, where it is usually expensive or even impossible to get enough high-quality training data. To address the problem, deep reinforcement learning (RL) is adopted to design motion planning for multirotors, which learns optimal mapping from states to actions directly using the trial-and-error training, so as to achieve efficient learning without demonstrations. For example, in [25], a deep RL algorithm is proposed for vision-based flight of quadrotors, where simulation and real-world experiences are integrated to achieve hybrid training. Aiming at achieving flexible multirotor navigation in indoor environments, memory-based deep RL is proposed to ensure obstacle avoidance [26]. These RL methods are proposed to learn optimal motion planning policy through trial-and-error interaction, but the complicated dynamics of multirotors and model-reality differences prevent obtaining safe and efficient algorithms. As a result, most RL is only effective in simulation environments, and the spatial-temporal optimality is not focused on, thereby generating reality-mismatched solutions.

After summarizing the existing methods for multirotor motion planning, it is found that some urgent problems remain unresolved as follows.

1) Solving nonconvex optimization of multirotor systems is challenging, making the motion planning inefficient, with spatial-temporal optimizations being urgent problems to be solved.
2) The gap between theory and practice exists for learning-based methods, and it is an urgent demand to exploit practice-oriented deep RL algorithm.
3) Few solutions solve the efficient cooperation problem of optimization and learning-based methods, poor generalization ability, low efficiency, and flexibility widely exist in multirotor motion planning.

In order to address these problems, a virtual goal-oriented hierarchical motion planning framework is proposed for multirotors, where the efficiency, smoothness, and flexibility are achieved by the deep integration of upper-layer learning and motion optimization. In this manner, the upper layer is designed to handle the nonconvex decision-making problem, and the lower layer is able to generate dynamically feasible and flexible navigation trajectories. The main contributions of this article are summarized as follows.

1) A novel virtual goal oriented hierarchical motion planning strategy is proposed for multirotors, where the upper-layer deep RL decision-making and lower-layer motion optimization are deeply integrated to achieve efficient learning and satisfactory navigation performance.
2) The virtual goal oriented deep RL is designed by fully considering the states and spatial path, where optimal commands are generated in real time, guaranteeing safe, efficient, smooth, and flexible navigation even in obstacle-rich environments.
3) A number of comparative experiments are implemented to verify the superior performance of the proposal in terms of navigation efficiency, smoothness, and flexibility.

The rest of this article is organized as follows. Section II briefly introduces the model of multirotors and the motion planning problem, the proposed strategy is provided in Section III. Then, experimental validation is given in Section IV. Finally, Section V concludes this article.

Notation: Throughout this article, these acronyms are adopted: virtual goal (VG), reinforcement learning (RL), quadratic programming (QP), minimum snap (MS), local variant minimum snap (LS), robot operating system (ROS), rough autoencoder (RAE), interval probability distribution learning (IPDL).

II. Problem Formulation

The objective of this article is to navigate the multirotor system to the target points, guaranteeing safe, efficient, smooth, and flexible travels in unknown environments as shown in Fig. 1. To this end, the mathematical model is first introduced, where the multirotor is modeled as a rigid body, and the following dynamic equation is obtained:

$$\begin{aligned} \dot{\boldsymbol{p}} &= \boldsymbol{v}, \dot{\boldsymbol{v}} = \frac{1}{m}\left(fR\boldsymbol{e}_3 - mg\boldsymbol{e}_3\right) \\ \dot{R} &= R\hat{\boldsymbol{\omega}}, \dot{\boldsymbol{\omega}} = \mathbb{I}^{-1}\left(\boldsymbol{\tau} - \boldsymbol{\omega} \times \mathbb{I}\boldsymbol{\omega}\right) \end{aligned} \tag{1}$$

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on December 02,2025 at 07:17:45 UTC from IEEE Xplore. Restrictions apply.

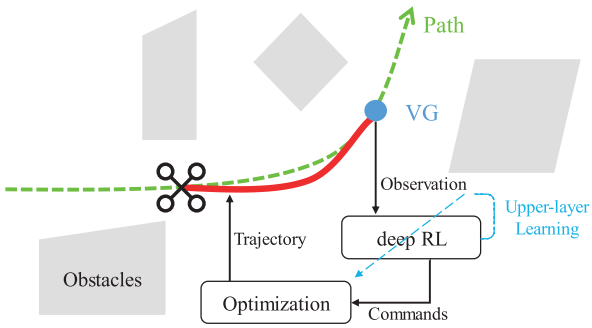


Fig. 1. Schematic illustration of VG-oriented learning-based motion planning systems.

where $\boldsymbol{p} \in \mathbb{R}^3$ and $\boldsymbol{v} \in \mathbb{R}^3$ represent the multirotor position and velocity in the inertial frame, respectively. $m \in \mathbb{R}$ denotes the multirotor mass, and $g \in \mathbb{R}$ stands for the gravity constant. $f \in \mathbb{R}$, $R \in \mathrm{SO}(3)$, and $\boldsymbol{e}_3 = [0\ 0\ 1]^\top \in \mathbb{R}^3$ denote the thrust input, the rotation matrix from the body-fixed frame to the inertial frame, and a unit vector, respectively. SO(3) stands for the special orthogonal group, which is defined as $\mathrm{SO}(3) = \{R \in \mathbb{R}^{3\times3} \mid R^\top R = I, \det R = 1\}$. $\boldsymbol{\omega} \in \mathbb{R}^3$ denotes the multirotor angular velocity in the body-fixed frame. $\mathbb{I} \in \mathbb{R}^{3\times3}$ is the multirotor inertia matrix, and the vector $\boldsymbol{\tau} \in \mathbb{R}^3$ denotes the torque input of multirotors.

To complete the navigation task in unknown environments, mapping and path searching algorithms are necessary, which is not the focus of this article, and typical mapping [27] and path searching [28] methods are adopted here. First, it is found from model (1) that the multirotor system is differentially flat. On this basis, the motion planning design can be carried out in the flat space, where all the states and inputs can be equivalently expressed in the flat space, avoiding processing the complicated high-dimensional state couplings directly. More details can be found in [14] and [16].

III. Hierarchical Motion Planning

The proposed framework is made up of three parts, named as training module, upper and lower layers, respectively, as shown in Fig. 2. Specifically, the framework is driven by online observation, where the training module samples all the states and rewards to update the upper-layer network, using the historical experiences to train the decision-making policy. In the framework, deep learning and optimization techniques are both considered to design the real-time motion planning algorithm, and the main results are proposed in the following three sections, named as virtual goal generation, upper-layer learning, and lower-layer optimization, respectively.

A. Virtual Goal Generation

For multirotors, the complicated dynamics, together with the unknown environments, brings much difficulty in real-time accurate motion planning. To address the problem, VG-guided hierarchical motion planning algorithm is proposed, where upper and lower layer policies are designed accordingly to achieve reliable navigation. Specifically, instead of directly generating feasible trajectories along the entire path, the dynamic VG is introduced in the framework as shown in Fig. 1, thereby ensuring

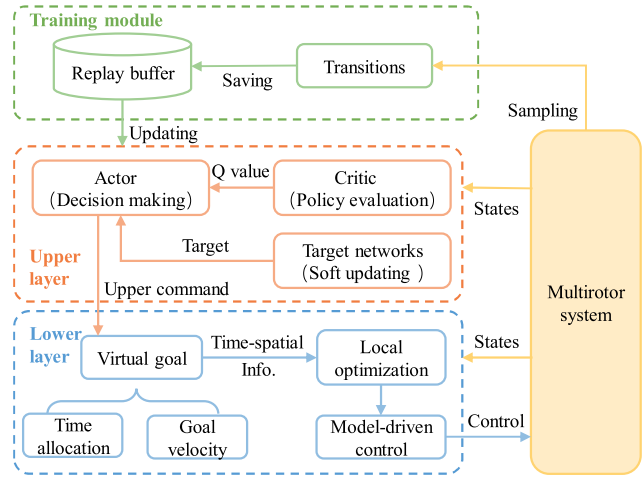


Fig. 2. Schematic illustration of the hierarchical motion planning framework.

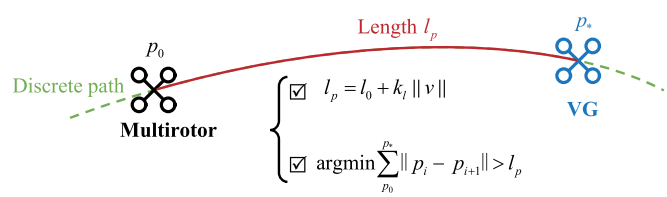


Fig. 3. Illustration for virtual goal generation process.

a low computation burden and real-time calculation. To obtain VG, the discrete path is searched from $\boldsymbol{p}_0$, current position, until the path length is larger than l_p, as shown in Fig. 3. In this manner, the VG is denoted as $\boldsymbol{p}_*$ in the path, which is online planned. To be more specific, the dynamic local range l_p is determined by $l_p = l_0 + k_l\|\boldsymbol{v}\|$, where $l_0 \in \mathbb{R}^+$ denotes the static value, and $k_l \in \mathbb{R}^+$ represents positive control gains, and $\| \star \|$ stands for 2-norm operator. To achieve satisfactory performance, it is necessary to take the safety assurance, smoothness, and spatial optimality into consideration to set proper l_p. Even after careful offline adjustments, the fixed value l_p leads to poor results in different environments. To address the problem, a dynamic l_p selection strategy is proposed, where the navigation velocity is considered to online tune the value. In this manner, the static l_0, together with the dynamic term $k_l\|\boldsymbol{v}\|$, guarantees satisfactory performance, where the high speed flight leads to a large VG length to ensure enough smoothness and prediction, and the low speed flight results in a small VG length to guarantee sufficient accuracy and flexibility.

Please note that the parameters l_0 and k_l are determined by the perception range of sensors and the maneuverability of multirotors. Specifically, larger l_0 tends to generate smoother navigation curves by sacrificing spatial performance, and smaller l_0 leads to exact navigation according to searched path by sacrificing dynamically feasible performance. Then, the dynamic term parameter k_l represents the intensity of velocity-driven adjustment, which is determined by the navigation aggressiveness of the multirotors. Aggressive flight requires larger k_l to guarantee flexible maneuvering, but too larger k_l may result in spatial optimality degradation. In practice, the navigation safety, flight

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on December 02,2025 at 07:17:45 UTC from IEEE Xplore. Restrictions apply.

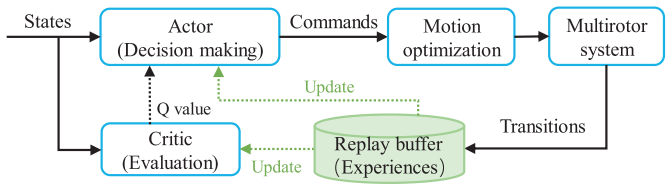


Fig. 4. Schematic illustration of upper-layer learning.

smoothness and velocity are fully evaluated to set proper l_0 and k_l.

Due to the control error, model-reality differences, mapping uncertainties, and so on, the current position of the multirotor may be far away from the path. Then, the current position $\boldsymbol{p}_0$ is unavailable, which leads to the failure of the proposed VG generation algorithm. To address the problem, we relax the $\boldsymbol{p}_0$ condition by adopting the nearest point in the path. Specifically, $\boldsymbol{p}_{\text{near}}$ point is determined in real-time following $\boldsymbol{p}_{\text{near}} = \text{argmin}\|\boldsymbol{p} - \boldsymbol{p}_i\|, \forall \boldsymbol{p}_i \in \xi$, where the set ξ contains all the points in the path, and the corresponding minimum distance from current multirotor position to the reference path is represented as $d_1 \in \mathbb{R}$. In this manner, the discrete path searching starts from $\boldsymbol{p}_{\text{near}}$ point, which tolerates tracking errors, making the planning feasible even if the current position is far away from the path. To this end, the VG is dynamically calculated following:

$$\text{argmin} \sum_{\boldsymbol{p}_{\text{near}}}^{\boldsymbol{p}_*} \|\boldsymbol{p}_i - \boldsymbol{p}_{i+1}\| > l_p. \tag{2}$$

In this way, the VG point $\boldsymbol{p}_*$ is searched along the path, until the path length exceeds l_p, and on this basis, the local trajectory is online generated using the VG.

We adopt an actor-critic network in this article, mainly because the policy gradient and value estimation are implemented in the networks [2], [19], [29]. Specifically, RL interacts with environments, and the learning is driven by maximizing cumulative reward. In this manner, the actor outputs planning action while the critic evaluates the actor policy [2]. In the future, we will work on integrating RAE, IPDL [30], [31] and deep RL technologies with each other to jointly improve the flight performance and robustness of multirotors.

B. Upper-Layer Learning

In the VG guided hierarchical motion planning framework, the upper-layer deep RL algorithm is responsible for precise decision-making as shown in Fig. 4. In the proposal, the input of the decision-making network is $\boldsymbol{s}_t = [\|\boldsymbol{v}\|, t_{\text{in}}, d_1, v_{\text{in}}] \in \mathbb{R}^4$ and the output is designed as $\boldsymbol{a}_t = [t_{\text{out}}, v_{\text{out}}] \in \mathbb{R}^2$, where t_{in} and v_{in} stand for the observation of current states, t_{out} represents the time allocation along the VG trajectory, and v_{out} stands for the desired velocity in the VG, representing the temporal and spatial optimality, respectively. Random exploration is adopted during training, where the action is linearly superimposed with zero-mean Gaussian noise $\boldsymbol{\epsilon} \in\sim \mathcal{N}(0, \sigma)$. For the reward design, the objective is to achieve safe and efficient navigation. To this end, the reward is proposed to increase the navigation velocity and decrease the nearest distance as $r_t = -k_d d_1^2 + k_v \|\boldsymbol{v}\|^2$, where k_d and k_v are positive constants. On this basis, the discounted reward is defined as $R_t = \sum_{i=0}^{+\infty} \gamma^i r_{t+i}$, where $\gamma \in (0, 1)$ stands for a discounted factor. To process, define the objective function $J(\pi)$ as $J = \mathbb{E}[R_1|\pi]$, and the gradient descent with respect to policy parameter α_1 is calculated following DPG theorem $\nabla_{\alpha_1} J(\pi_{\alpha_1}) = \mathbb{E}_{\boldsymbol{s}\sim\rho^\pi}[\nabla_{\alpha_1}\pi(\boldsymbol{s})\nabla_{\boldsymbol{a}} Q^\pi(\boldsymbol{s}, \boldsymbol{a})|_{\boldsymbol{a}=\pi(\boldsymbol{s})}]$, where the decision-making algorithm is updated following the gradient decent, and the action-value function Q^π is updated based on the temporal difference learning.

Algorithm 1: Upper-Layer Training Algorithm.

```
Randomly initialize the actor-critic networks π, Q1, Q2.
Initialize target networks following π, Q1, Q2.
Initialize replay buffer B.
for episode i = 1, ··· N1 do
  Initialize the multirotor states.
  Load the generated path.
  for step j = 1, ..., N2 do
    Generate the VG point along the path following (2).
    Generate upper-layer decision-making action a_t.
      Execute decision, lower-layer planning and
      control.
    Detect states s_{t+1} and calculate reward r_t.
    Save the transition in B.
    if B is full then
      Update networks by sampling data from B.
    end if
  end for
end for
```

The training process is presented in Algorithm 1. First of all, the double critics Q_1 and Q_2 are introduced to independently evaluate the decision-making policy π. Then, the target networks π', Q_1' and Q_2' are put forward to increase the learning stability, which generates the target action-value function $y_i = r_i + \gamma \min_{j=1,2} Q_j'(\boldsymbol{s}_{i+1}, \pi'(\boldsymbol{s}_{i+1}) + \boldsymbol{\epsilon}_2)$, with i denoting the data sequence subscript, j standing for the critic number. Noticing that the nearby states should generate similar decisions, the zero-mean Gaussian noise $\boldsymbol{\epsilon}_2 \sim \mathcal{N}(0, \sigma_2)$ is adopted in the target action-value function to smooth the value estimation and ease the chattering problem of upper-layer decision-making. On this basis, the target networks are online updated according to the original networks in a soft manner $\alpha_i' = (1 - \eta)\alpha_i' + \eta\alpha_i$, where the soft parameter satisfies $0 < \eta \ll 1$, and α_i denote the weights of actor and critics, respectively. To decrease the correlations between training samples, the replay buffer $\mathcal{B}$ is designed to store training experiences. It is noted that if the replay buffer is full, the oldest data will be replaced with the newly sampled data, where the first-in-first-out rule is adopted in constructing the replay buffer. During training, the important parameter, buffer size, is designed to make sure that the replay buffer experiences are all visited in the training. In this manner, it is guaranteed that the important data are already learned before being replaced, and the data importance affects the training slightly, which is not considered here. By employing the experience replay technique, the correlations between training samples are decreased efficiently. Then, the training is carried out by randomly sampling data from the replay buffer $\mathcal{B}$, where the gradient descent is adopted to update the policy as $\alpha_1(t+1) = \alpha_1(t) + \lambda_{\alpha_1}\nabla_{\alpha_1} J(\pi_{\alpha_1})$. The critic networks are updated following $\alpha_i(t+1) = \alpha_i(t) + \lambda_{\alpha_i}\nabla_{\alpha_i} L_j, \ \ i = 2, 3$, where $\lambda_{\alpha_i} \in \mathbb{R}$ denote positive learning rates, and $L_j = \frac{1}{M}\sum_{k=1}^{M}[y_k - Q_j(\boldsymbol{s}_k, \boldsymbol{a}_k)]^2, \ \ j = 1, 2$ denote

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on December 02,2025 at 07:17:45 UTC from IEEE Xplore. Restrictions apply.

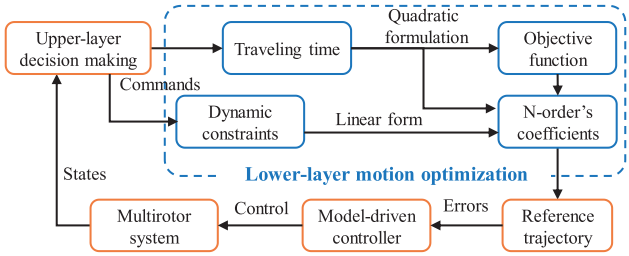


Fig. 5. Schematic illustration of lower-layer motion optimization.

the loss functions of the critic networks, with $M \in \mathbb{R}$ standing for the batch size of sampling. This completes the upper-layer learning design.

C. Lower-Layer Optimization

According to the upper-layer commands, the lower-layer optimization and control are implemented in this section. Specifically, the VG point $\boldsymbol{p}_*$ is obtained following argmin $\sum_{\boldsymbol{p}_{\text{near}}}^{\boldsymbol{p}_*} \|\boldsymbol{p}_i - \boldsymbol{p}_{i+1}\| > l_p$, and the corresponding spatial-temporal optimality is decided by the upper-layer RL algorithm, where the traveling time t_{out} and goal velocity v_{out} are generated in real time. On this basis, the lower-layer optimization is designed in a two-step process as shown in Fig. 5. First, the polynomial function is utilized to parameterize the planning trajectory, where the coefficients of the function is online optimized to achieve optimal results. Second, the minimum-snap objective function, together with the spatial-temporal constraints, is designed to formulate the optimization problem, which can be solved efficiently using the QP solver. Specifically, the n-order polynomial function is defined as $\boldsymbol{p}_d = \sum_{i=0}^{n} \boldsymbol{c}_i t^i$, with $\boldsymbol{c}_i \in \mathbb{R}^3$ representing the coefficient vectors, and the trajectory time t satisfies $t \in (t_0, t_0 + t_{\text{out}})$. Then, the local motion planning problem is formulated in the minimum-snap manner, where the objective function is designed as $J = \sum_{i=x,y,z} \int_{t_0}^{t_0+t_d} (\frac{\mathrm{d}^4 p_{di}}{\mathrm{d}t^4})^2 \mathrm{d}t$, with the snap-level derivative being adopted in optimizing the final results. On this basis, the quadratic formulation $J = \boldsymbol{H}^\top Q \boldsymbol{H}$ can be formulated, where $\boldsymbol{H}$ denotes the vector containing coefficients of polynomial in the x, y, z-axes, and Q stands for the Hessian matrix of the objective function. The transformation process can be found in [14], which is hence omitted here. Then, the spatial-temporal constraints are imposed on the quadratic formulation, which is compiled into a linear equality constraint $A\boldsymbol{H} = \boldsymbol{B}$. In this way, the lower-layer optimization is carried out to obtain real-time trajectories. Based on the optimization results, the controller is designed following the outer–inner-loop frames [32], where the outer-loop virtual force is designed as $\boldsymbol{f}_d = mg\boldsymbol{e}_3 + m\ddot{\boldsymbol{p}}_d - K_1\dot{\boldsymbol{e}}_p - K_2\boldsymbol{e}_p - K_3 \int_0^t \boldsymbol{e}_p \mathrm{d}\tau$, and inner-loop torque is designed as $\boldsymbol{\tau} = -K_a\boldsymbol{\chi} - K_b\boldsymbol{e}_R - \boldsymbol{\tau}_f$, where K_1, K_2, K_3 and K_a, $K_b \in \mathbb{R}^{3\times 3}$ are positive definite matrices, and $\boldsymbol{\tau}_f$ denotes the feedforward term in the attitude control. The proposed controller outputs motion-level signals to the multirotors, where the Lyapunov-based analysis can be carried out to prove the asymptotic convergence.

Remark 3: It is noted that there exist significant differences between the proposed method and our previous work [33].

1) Motion planning framework: The virtual goal (VG) is first designed in this article, where the VG point is calculated according to the spatial path and current velocity. 2) Decision-making policy. The actor-critic framework is adopted in this article, where the VG oriented learning achieves accurate decision-making on the spatial-temporal optimality.

IV. Experimental Validation

In this section, the motion planning performance of the proposed method is validated by conducting a series of simulation and experimental experiments. In particular, the proposed scheme is first trained in the Gazebo environment [34], where all the communications are implemented using ROS on the Ubuntu 18.04 system. Then, the obtained strategy is tested by setting different environment. On this basis, a homemade multirotor platform, equipped with onboard camera and minicomputer, is further utilized to validate the effectiveness of the proposed method in real-world experiments.

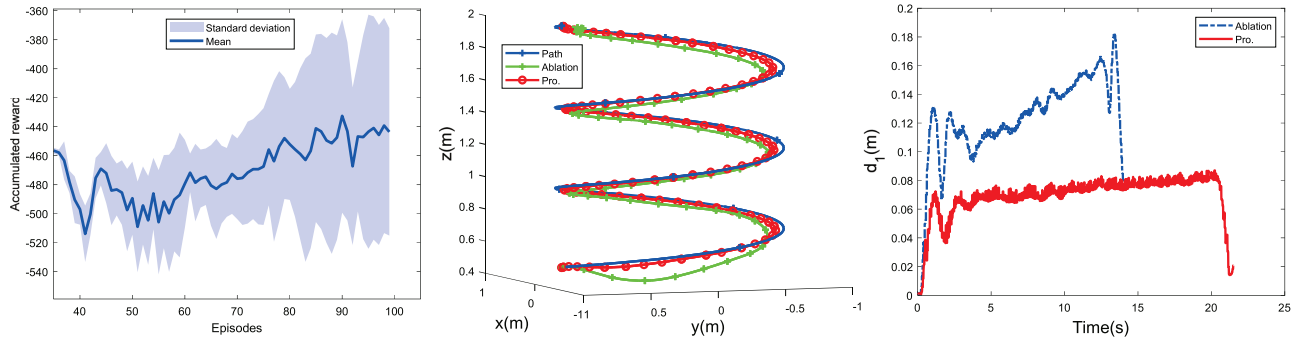
A. Training and Simulation

The proposed method is trained in simulation in training phase, then, the trained policy is deployed in experiments in deployment phase. Specifically, to train the proposed deep RL motion planning policy, we construct the simulation environments, where the Pelican multirotor provided by RotorS simulator, is used to generate data in Ubuntu 18.04 system using the Gazebo engine, and the interaction experiences are utilized to update the policy. After training, the obtained policy is deployed in a homemade quadrotor.

During training, the perception is achieved by adopting a depth camera, where the point-cloud output is fed into Octomap algorithm [27], generating grid map in real time. D* algorithm [28] is utilized to search for feasible paths online, where the VG point is generated along the path in real time. The hyperparameter values are tuned according to the multirotor dynamics and physical meaning, where the efficiency, convergence rate, and stability are considered. On this basis, the main training parameters are decided as follows. The mass of multirotor is $m = 2.2$ kg, the inertia is $\mathbb{I} = \text{diag}[0.1\ 0.1\ 0.1]$ kg$\cdot$ m^2. The reward parameters are set as $k_d = 0.1$ and $k_v = 1.0$, respectively, with the discounted factor being set as $\gamma = 0.95$. The learning rates are chosen as $\lambda_{\alpha_1} = 10^{-4}$ and $\lambda_{\alpha_2} = \lambda_{\alpha_3} = 2 \times 10^{-4}$, with the soft parameter $\eta = 0.001$ and batch size $M = 4000$. The training episodes and steps are set as $N_1 = 100$ and $N_2 = 200$, respectively, with sampling size being chosen as 128. The upper-layer deep RL and lower-layer optimization run at 10 Hz, and the flight control is implemented at 50 Hz. The positive constant l_0 is set as 0.6, and the dynamic gain k_l is chosen as 0.2. Then, the obtained policy is tested in various different scenarios, including Gazebo and indoor tests, to verify the effectiveness of the proposed method.

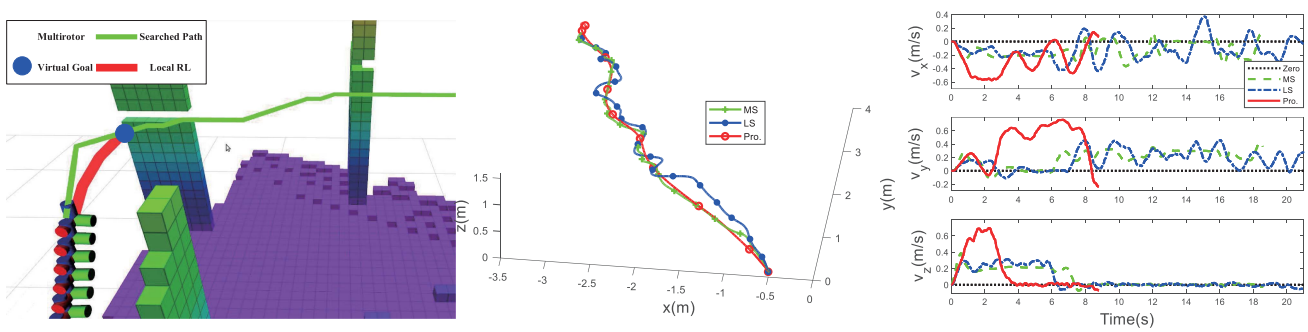
To show the effectiveness of the hierarchical navigation framework, we have done additional learning. Specifically, the optimization layer is removed, and the deep RL is designed to output the motion-level signals. We have done the training repeatedly and ten trial results are plotted in Fig. 6. It is seen that the accumulated reward fluctuates seriously. For one thing, the mean value of accumulated reward does not converge. For another, the standard deviation of the repeated trials is divergent, which means that every training trial is updated differently, indicating that the learning is not able to find a feasible solution.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on December 02,2025 at 07:17:45 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c)

Fig. 6. (a) Accumulated reward of ten trials using only deep RL. (b) Navigation trajectories of the proposed and ablation methods. (c) Minimum distance d_1 from the multirotor to the reference path.



(a) (b) (c)

Fig. 7. Motion planning results in test 1. (a) Snapshot of the generated VG and local RL trajectory. (b) Traveling trajectories comparison. (c) Navigation velocity of the three methods.

In addition, to verify the effectiveness of the reward design, we have done the ablation experiments by removing the penalty term $-k_d d_1^2$. The trained policy is applied for tests, where the path $x = 0.8\sin(6\pi s)$ m, $y = 0.8\cos(6\pi s)$ m, $z = 1.4997s + 0.5$ m, is adopted for comparison, with $s \in [0, 1]$ being the path coordinate. We use a fixed path for comparison to show the results intuitively. It is seen from Fig. 6 that the comparative results show much larger path errors, which indicates that the navigation spatial performance is poor, and it may lead to infeasible results or collisions in obstacle-rich environments. To better show the spatial performance comparison, the minimum distance from multirotor position to the reference path is plotted in Fig. 6(c). It is observed that the navigation curve of the comparative one is far from the given path, which is larger than 0.18 m.

Then, the obtained policy is tested in the Gazebo environments, where the obstacle distribution, initial and target points in Tests 1,2,3, are shown in Fig. 8. To show the effectiveness of the proposed motion planning method (Pro.), the optimization-based minimum snap [14] (MS), and its local variant (LS) are adopted for comparison. It is noted that MS is constructed following [14], where the global path is utilized to generate the navigation trajectory, but the LS is designed following the local replanning frame, which generates local trajectories in a fixed range [24]. The two methods are adopted for comparison, mainly considering that the proposal achieves efficient motion planning by the VG guided learning in a local manner, and the local and global planning methods are widely used so far. By the comparison, we intend to show the effectiveness of the proposed learning-based motion planning framework.

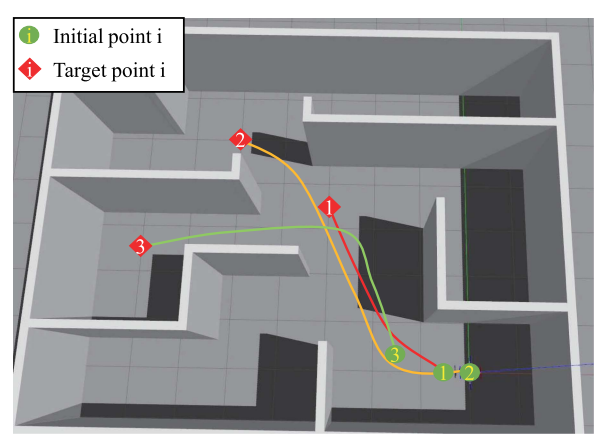


Fig. 8. Obstacle distribution, initial and target points in Tests 1,2,3 in Gazebo environment.

1) Test 1: In test 1, the initial and target points are set as $[-0.5\ 0\ 0.1]^\top$ m and $[-3\ 3.5\ 1.5]^\top$ m, respectively. Three methods are implemented in this section, and the obtained results are provided in Fig. 7. Specifically, the navigation of the proposed

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on December 02,2025 at 07:17:45 UTC from IEEE Xplore. Restrictions apply.

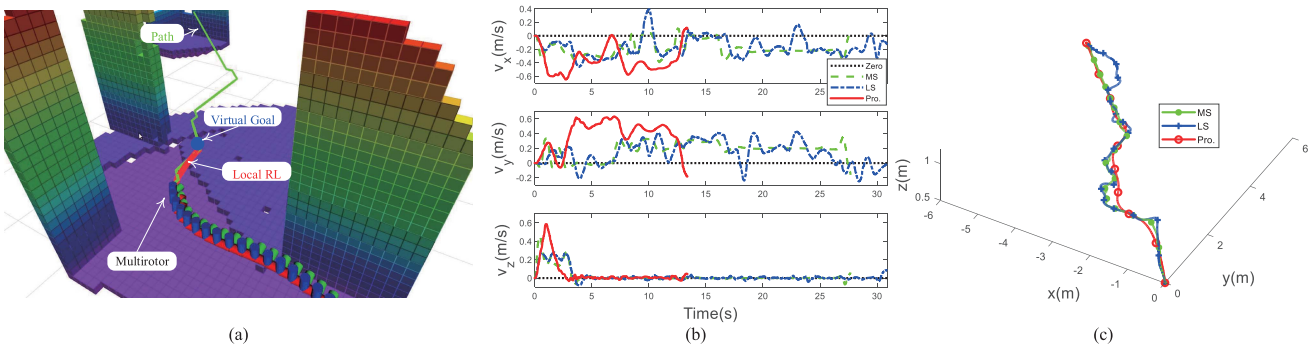


Fig. 9. Motion planning results in test 2. (a) Snapshot of the virtual goal oriented navigation. (b) Traveling velocity comparison. (c) Navigation trajectories of three methods.

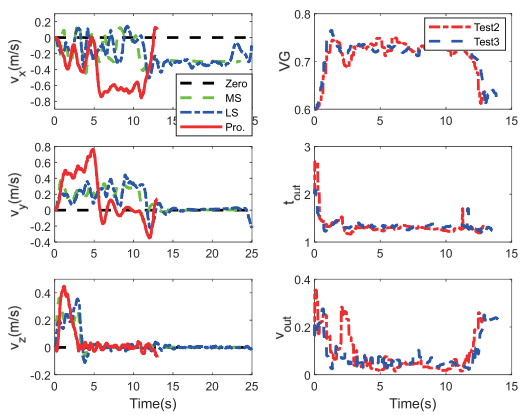


Fig. 10. Upper-layer command curves in tests 2 and 3, and traveling velocity in test 3.

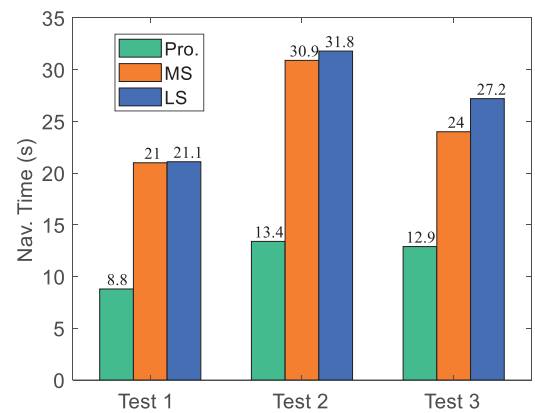


Fig. 11. Navigation time comparison of tests 1, 2, and 3.

method is achieved by online generating a virtual goal and local RL trajectory as shown in Fig. 7(a). The flight paths of three methods are recorded in Fig. 7(b), where the proposed method performs much better than the comparative methods. Besides, the velocities in three axes are plotted in Fig. 7(c), and the efficient navigation of the proposed method is well demonstrated by the comparison, where much faster traveling velocities are obtained during the test. Based on the results shown in Fig. 7, it is concluded that efficient and accurate navigation is guaranteed by the proposed method, and its performance is superior over the comparative methods.

2) Test 2: In this section, the initial and target points are changed as $[0\ 0\ 0.5]^\top$ m and $[-5\ 5\ 1.2]^\top$ m, and the motion planning parameters are the same as those in test 1. The obtained results are recorded in Figs. 9–11. It can be observed from Fig. 9 that the proposed method performs much better in terms of navigation velocity and efficiency. Benefiting from the designed intelligent hierarchical motion planning framework, the proposal completes the navigation task fastest, where the local trajectories are online generated to navigate the multirotor. Besides, the virtual goal is plotted in Fig. 9(a), which is dynamically changed according to current velocity and spatial path, thereby guaranteeing satisfactory smoothness and flexibility. The navigation trajectory in test 2 is plotted in Fig. 9(c), and it is found that the proposed method navigates the multirotor much smooth and flexible. It is also seen from Fig. 10 that, the upper-layer outputs t_{out} and v_{out} are automatically changed according to current states. During the test, the RL trajectories are frequently generated, where the spatial-temporal optimality is guaranteed by the upper-layer decision-making policy and lower-layer optimization.

3) Test 3: To further validate the effectiveness and generalization ability of the proposed method, test 3 is carried out by adopting the initial and target points $[-1.5\ 0.3\ 1.2]^\top$ m and $[-7\ 2.5\ 1.8]^\top$ m, respectively. The obtained results are plotted in Figs. 10 and 11, where the navigation velocity is recorded in the left column of Fig. 10, and the upper-layer decision-making results are given in the right column. Besides, the flight time in three tests is recorded in Fig. 11, and it is observed that the proposed method performs much better than the comparative methods in all tests, where the shortest navigation time is always obtained by the proposal. It is seen that the proposed method drives the multirotor in unknown environments efficiently, which achieves much faster navigation velocities in three axes, and the traveling time is also much smaller than the comparative methods. These experiment results directly demonstrate that, by designing the virtual goal oriented RL framework, enhanced multirotor motion planning is achieved, and consequently, superior navigation performance is observed in the experiment.

In addition, we have done additional tests by comparing it with iterative optimization (IO). Specifically, IO is widely adopted to solve the nonconvex planning problem by iterative time

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on December 02,2025 at 07:17:45 UTC from IEEE Xplore. Restrictions apply.

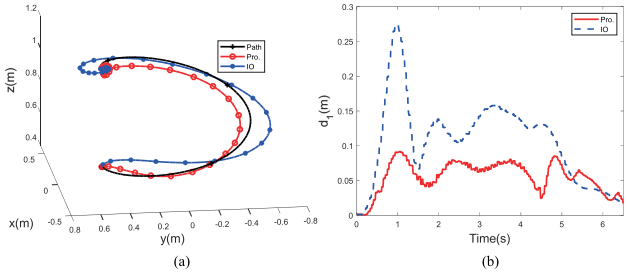


Fig. 12. Additional test. (a) Navigation trajectories of IO and the proposed methods. (b) Minimum distance d_1 from the multirotor to the reference path.

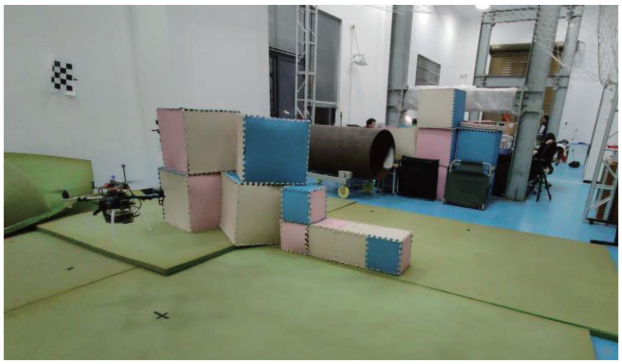


Fig. 13. Snapshot of the navigation experiment in unknown environments.

refinement [22], [35]. To this end, we perform the comparison to verify the temporal-spatial optimality of the proposed method. It is seen from Fig. 12 that the navigation task is completed by the IO and the proposed methods, taking almost the same amount of time, where the minimum distance d_1 of the IO method is much larger than the proposed one. On this basis, it is concluded that the proposed method navigates multirotors with better temporal and spatial performances.

B. Sim-to-Real Transfer

In order to verify the effectiveness of the proposed virtual goal oriented deep RL framework, a physical quadrotor platform is constructed to implement the proposal. Specifically, the quadrotor is made up of a mechanical body, a PX4 autopilot, an Intel D435i depth camera, and a NUC-based calculation system as shown in Fig. 13. Specifically, the multirotor platform is a typical X450 quadrotor with a motor wheelbase of 450 mm. The multirotor mass is 2.2 kg, and its moment of inertia is $\mathrm{diag}[0.01\ 0.01\ 0.02]\ \mathrm{kg}\cdot\mathrm{m}^2$. Please note that after about tens of thousands of trainings, the proposed policy converges to feasible and efficient solution. Then, the obtained policy is implemented in real-world multirotors without any modification. To achieve sufficient verification, two groups of hardware experiments are carried out in this section, where the initial and target points, and obstacle configuration are all randomly set. The effectiveness and generalization ability of the proposed method are verified by the comparisons with the MS method [14], where the LS one is not implemented here for safety, considering the worse performance in previous simulation tests.

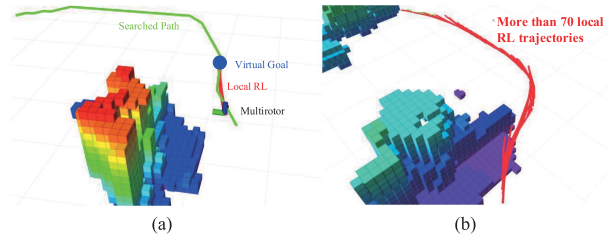


Fig. 14. Generated trajectories in the case 1. (a) Snapshot of the local RL trajectory. (b) All RL trajectories.

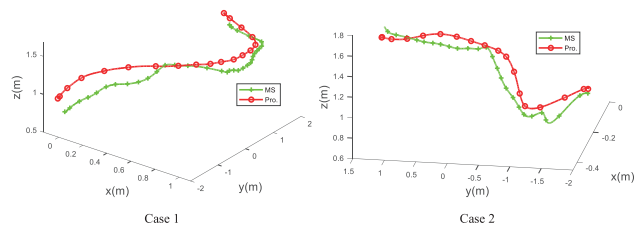


Fig. 15. Navigation trajectories of MS and Pro. methods in cases 1 and 2.

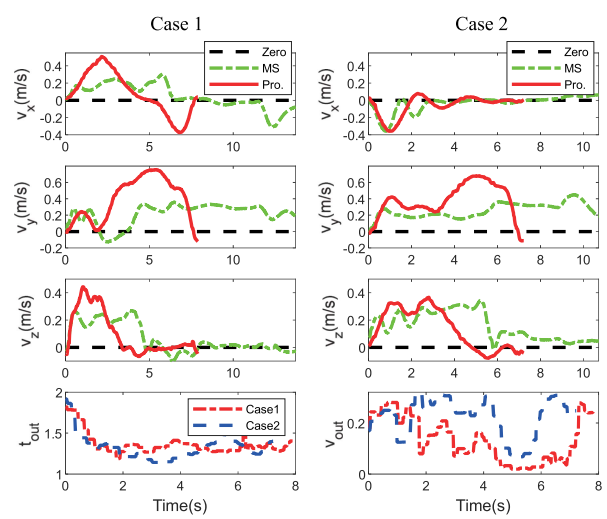


Fig. 16. Traveling velocity and upper-layer command curves in cases 1 and 2.

1) Case 1: In case 1, the obstacles are set as shown in Fig. 13, where the initial and target points are chosen as $[0\ \ -1.5\ \ 0.8]^\top$ m and $[0.6\ \ 1.5\ \ 1.6]^\top$ m, respectively. The two methods can navigate the quadrotor successfully to the target point. Specifically, the experimental results for the proposed and the MS methods are plotted in Figs. 14–16. It is found from Fig. 14 that in unknown environment, the grid map is constructed successfully, and the searched path and current states of quadrotor are utilized to generate virtual goal in real time, which leads to the local RL trajectory generation using the proposed hierarchical motion planning. In case 1, more than 70 local RL trajectories are generated according to the RL policy, which navigates the multirotor to the target point efficiently, smoothly, and flexibly. In addition, the traveling curves are plotted in Fig. 15, where the proposal drives the quadrotor much smoother than the comparative one. The advantage of the proposal is that the temporal and spatial performance are both considered in the design.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on December 02,2025 at 07:17:45 UTC from IEEE Xplore. Restrictions apply.

TABLE I
FLIGHT TIME IN TWO EXPERIMENTAL CASES

	Case 1		Case 2	
Method	Pro.	MS	Pro.	MS
Time (s)	**7.8**	17.3	**7.2**	15.78

2) Case 2: In this case, the obstacles, initial and target points are all set differently, where the obstacles' configuration is shown in the video, and $[0.1 \ \ -1.7 \ 0.9]^\top$ m and $[-0.3 \ 1.2 \ 1.8]^\top$ m are the initial and target points. The navigation curves of the two methods in case 2 are plotted in Fig. 15, and it is observed that even in different environment, the proposed method can drive the quadrotor successfully. Besides, the navigation velocity curves are plotted in Fig. 16, it is found from the velocity records that, in the two cases, the proposed method performs much faster navigation, and requires a shorter traveling time. To show the decision-making performance of the proposed method more clearly, the generated upper-layer commands are recorded in the fourth line of Fig. 16. It is seen from Table I and Fig. 16 that the upper-layer decision-making policy outputs spatial-temporal commands online to guide the quadrotor to complete navigation tasks. Benefiting from the proposed VG-oriented learning framework, the intelligent RL trajectories can be generated in real time, and the computation burden is small. Therefore, the best performance is obtained by the proposed method in all the tests.

The proposal is a practice-oriented design. The remarkable feature of this article is that the data-driven upper-level learning and optimization-driven motion-level computation are integrated in the point-to-VG planning, which navigates multirotors efficiently even in computation resource-limited onboard chips. In this way, the learning is carried out in the upper layer, which tries to tune the spatial and temporal performances. Thus, the differences between simulation and real-world application are not involved directly, making the learning policy reliable in real-world application. Then, the lower layer optimization and control are designed to reduce the model-reality mismatches, guaranteeing satisfactory real-world performance in the sim-to-real transfer section.

V. CONCLUSION

In this article, an intelligent hierarchical motion planning strategy was proposed for multirotors, which achieved reliable, flexible, and efficient navigation in unknown environments. By observing real-time states, the VG was online-generated along the global path utilizing the spatial path and velocity signals. On this basis, deep RL was designed to achieve upper-layer learning, which dealt with the complicated spatial-temporal decision-making problem. In this manner, after trail-and-error learning, the deep RL policy outputs reliable decisions in real time, which were fed into the lower-layer module, guiding the motion optimization and control. Different from traditional motion planning strategies, the VG guided upper-layer deep learning and lower-layer optimization, control were integrated to provide satisfactory solutions in real time. Besides, instead of calculating all trajectories directly, the deep RL focused on the VG, where the local computation of time allocation and spatial distribution were much efficient and flexible compared with other learning-based solutions. Comparative experiments were included to demonstrate the superior navigation performance of the proposed hierarchical motion planning strategy, and its feasibility, efficiency, navigation smoothness, and flexibility were fully validated by a number of comparison studies. Nevertheless, the performance depended largely on accurate measurements, where the signals d_1 and $\boldsymbol{v}$ may be unavailable or inaccurate in extreme environments, leading to unsatisfactory navigation performance. In the future, we will work on intelligent and efficient motion planning for multirotors to solve the navigation problems without accurate measurements. Specifically, we intend to solve the problem by the combination of onboard RGB image and convolutional neural network, which learns end-to-end navigation policy from image data instead of using traditional mapping and localization.

REFERENCES

[1] S. Yang, B. Xian, J. Cai, and G. Wang, "Finite-time convergence control for a quadrotor unmanned aerial vehicle with a slung load," *IEEE Trans. Ind. Inform.*, vol. 20, no. 1, pp. 605–614, Jan. 2024.

[2] H. Hua and Y. Fang, "A novel reinforcement learning-based robust control strategy for a quadrotor," *IEEE Trans. Ind. Electron.*, vol. 70, no. 3, pp. 2812–2821, Mar. 2023.

[3] X. Zhang, X. Xu, Y. Liu, H. Wang, X. Zhang, and Y. Zhuang, "FGIP: A frontier-guided informative planner for UAV exploration and reconstruction," *IEEE Trans. Ind. Inform.*, vol. 20, no. 4, pp. 6155–6166, Apr. 2024.

[4] A. Nikhilraj, H. Simha, and H. Priyadarshan, "Optimal energy trajectory generation for a quadrotor UAV using geometrically exact computations on SE(3)," *IEEE Control Syst. Lett.*, vol. 3, no. 1, pp. 216–221, Jan. 2019.

[5] Z. Zhou, G. Wang, J. Sun, J. Wang, and J. Chen, "Efficient and robust time-optimal trajectory planning and control for agile quadrotor flight," *IEEE Robot. Autom. Lett.*, vol. 8, no. 12, pp. 7913–7920, Dec. 2023.

[6] Z. Liu and L. Cai, "Simultaneous planning and execution for quadrotors flying through a narrow gap under disturbance," *IEEE Trans. Control Syst. Technol.*, vol. 31, no. 6, pp. 2644–2659, Nov. 2023.

[7] H. Wang, S. Song, Q. Guo, D. Xu, X. Zhang, and P. Wang, "Cooperative motion planning for persistent 3D visual coverage with multiple quadrotor UAVs," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 3, pp. 3374–3383, Jul. 2024.

[8] X. Yi, B. Luo, and Y. Zhao, "Neural network-based robust guaranteed cost control for image-based visual servoing of quadrotor," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 12693–12705, Sep. 2024.

[9] H. Liu, X. Wang, and Y. Zhong, "Quaternion-based robust attitude control for uncertain robotic quadrotors," *IEEE Trans. Ind. Inform.*, vol. 11, no. 2, pp. 406–415, Apr. 2015.

[10] N. J. Sanket et al., "EVdodgeNet: Deep dynamic obstacle dodging with event cameras," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2020, pp. 10651–10657.

[11] J. Park, Y. Lee, I. Jang, and H. J. Kim, "DLSC: Distributed multi-agent trajectory planning in maze-like dynamic environments using linear safe corridor," *IEEE Trans. Robot.*, vol. 39, no. 5, pp. 3739–3758, Oct. 2023.

[12] J. Zeng, P. Kotaru, M. W. Mueller, and K. Sreenath, "Differential flatness based path planning with direct collocation on hybrid modes for a quadrotor with a cable-suspended payload," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 3074–3081, Apr. 2020.

[13] Y. Liu, Q. Wang, H. Hu, and Y. He, "A novel real-time moving target tracking and path planning system for a quadrotor UAV in unknown unstructured outdoor scenes," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 49, no. 11, pp. 2362–2372, Nov. 2019.

[14] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2011, pp. 2520–2525.

[15] Z. Wang, X. Zhou, C. Xu, J. Chu, and F. Gao, "Alternating minimization based trajectory generation for quadrotor aggressive flight," *IEEE Robot. Autom. Lett.*, vol. 5, no. 3, pp. 4836–4843, Jul. 2020.

[16] C. Richter, A. Bry, and N. Roy, *Polynomial Trajectory Planning for Aggressive Quadrotor Flight in Dense Indoor Environments*. Cham, Switzerland: Springer, 2016, pp. 649–666.

[17] C. Y. Son, H. Seo, D. Jang, and H. J. Kim, "Real-time optimal trajectory generation and control of a multi-rotor with a suspended load for obstacle avoidance," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 1915–1922, Apr. 2020.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on December 02,2025 at 07:17:45 UTC from IEEE Xplore. Restrictions apply.

[18] A. Hakobyan, G. C. Kim, and I. Yang, "Risk-aware motion planning and control using CVaR-constrained optimization," *IEEE Robot. Autom. Lett.*, vol. 4, no. 4, pp. 3924–3931, Oct. 2019.
[19] Y. Wang, J. Sun, H. He, and C. Sun, "Deterministic policy gradient with integral compensator for robust quadrotor control," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 50, no. 10, pp. 3713–3725, Oct. 2020.
[20] Y. Yang, S. Gorbachev, B. Zhao, Q. Liu, Z. Shu, and D. Yue, "Predictor-based neural attitude control of a quadrotor with disturbances," *IEEE Trans. Ind. Inform.*, vol. 20, no. 1, pp. 169–178, Jan. 2024.
[21] G. Chou, N. Ozay, and D. Berenson, "Uncertainty-aware constraint learning for adaptive safe motion planning from demonstrations," in *Proc. Conf. Robot Learn.*, 2021, pp. 1612–1639.
[22] M. M. d. Almeida, R. Moghe, and M. Akella, "Real-time minimum snap trajectory generation for quadcopters: Algorithm speed-up through machine learning," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2019, pp. 683–689.
[23] X. Zhang et al., "Relational navigation learning in continuous action space among crowds," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2021, pp. 3175–3181.
[24] A. Loquercio, E. Kaufmann, R. Ranftl, A. Dosovitskiy, V. Koltun, and D. Scaramuzza, "Deep drone racing: From simulation to reality with domain randomization," *IEEE Trans. Robot.*, vol. 36, no. 1, pp. 1–14, Feb. 2020.
[25] K. Kang, S. Belkhale, G. Kahn, P. Abbeel, and S. Levine, "Generalization through simulation: Integrating simulated and real data into deep reinforcement learning for vision-based autonomous flight," in *Proc. 9 IEEE Int. Conf. Robot. Autom.*, 2019, pp. 6008–6014.
[26] A. Singla, S. Padakandla, and S. Bhatnagar, "Memory-based deep reinforcement learning for obstacle avoidance in UAV with limited environment knowledge," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 1, pp. 107–118, Jan. 2021.
[27] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "Octomap: An efficient probabilistic 3D mapping framework based on octrees," *Auton. Robots*, vol. 34, no. 3, pp. 189–206, 2013.
[28] A. Stentz, "Optimal and efficient path planning for partially-known environments," in *Proc. IEEE Int. Conf. Robot. Autom.*, 1994, vol. 4, pp. 3310–3317.
[29] T. Zhang, G. Kahn, S. Levine, and P. Abbeel, "Learning deep control policies for autonomous aerial vehicles with MPC-guided policy search," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2016, pp. 528–535.
[30] Z. He, M. Wu, X. Zhao, S. Zhang, and J. Tan, "A generative feature-to-image robotic vision framework for 6D pose measurement of metal parts," *IEEE/ASME Trans. Mechatron.*, vol. 27, no. 5, pp. 3198–3209, Oct. 2022.
[31] J.-F. Toubeau, J. Bottieau, F. Vallée, and Z. De Grève, "Deep learning-based multivariate probabilistic forecasting for short-term scheduling in power markets," *IEEE Trans. Power Syst.*, vol. 34, no. 2, pp. 1203–1215, Mar. 2019.
[32] X. Zhang, Y. Zhuang, X. Zhang, and Y. Fang, "A novel asymptotic robust tracking control strategy for rotorcraft UAVs," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 4, pp. 2338–2349, Oct. 2023.
[33] H. Hua and Y. Fang, "A novel learning-based trajectory generation strategy for a quadrotor," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 7, pp. 9068–9079, Jul. 2024.
[34] F. Furrer, M. Burri, M. Achtelik, and R. Siegwart, "RotorS–A modular gazebo MAV simulator framework," in *Robot Operating System (ROS) The Complete Reference (Volume 1)*. Berlin, Germany: Springer, 2016, pp. 595–625.
[35] K. Liu et al., "Optimal time trajectory generation and tracking control for over-actuated multirotors with large-angle maneuvering capability," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 8339–8346, Jul. 2022.

Hean Hua received the B.Eng. degree in measurement and control technology and instruments from Harbin Engineering University, Harbin, China, in 2017, and the Ph.D. degree in control science and engineering from Nankai University, Tianjin, China, in 2022.

He is currently an Assistant Professor with the School of Robotics, Hunan University, Changsha, China, and also with the National Engineering Research Center for Robot Visual Perception and Control Technology, Hunan University, Changsha, China. His current research interests include motion planning, intelligent control and machine learning for autonomous multirotor UAVs.

Yaonan Wang received the B.S. degree in computer engineering from the East China University of Science and Technology, Fuzhou, China, in 1981 and the M.S. and Ph.D. degrees in control engineering from Hunan University, Changsha, China, in 1990 and 1994, respectively.

He was a Postdoctoral Research Fellow with the National University of Defense Technology, Changsha, from 1994 to 1995, a Senior Humboldt Fellow in Germany from 1998 to 2000, and a Visiting Professor with the University of Bremen, Bremen, Germany, from 2001 to 2004. He has been a Professor with Hunan University since 1995. He has been an academician of China Engineering Academy since 2019. His research interests include robot control, intelligent control and information processing, industrial process control, and image processing.

Hang Zhong (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in automation science from the College of Electrical and Information Engineering, Hunan University, Changsha, China, in 2013, 2016, and 2020, respectively.

From 2020 to 2022, he was a Postdoc Fellow with the Department of Electrical and Information Engineering, Hunan University, Changsha, China. He is currently an Associate Professor with the School of Robotics, Hunan University, Changsha, China. His research interests include aerial robotics, multi-robot systems, visual servoing, visual navigation, and nonlinear control.

Hui Zhang (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in pattern recognition and intelligent system from Hunan University, Changsha, China, in 2004, 2007, and 2012, respectively.

He is currently a Professor with the School of Robotics and the National Engineering Research Center for Robot Visual Perception and Control Technology, Hunan University. He was a Visiting Scholar with the Common Vulnerability Scoring System Laboratory, Department of Electrical and Computer Engineering, University of Windsor, Windsor, ON, Canada, in 2017. His research interests include machine vision, image processing, and robot vision.

Yongchun Fang (Senior Member, IEEE) received the B.S. degree in electrical engineering and the M.S. degree in control theory and applications from Zhejiang University, Hangzhou, China, in 1996 and 1999, respectively, and the Ph.D. degree in electrical engineering from Clemson University, Clemson, SC, USA, in 2002.

He is currently a Professor with the Institute of Robotics and Automatic Information System (IRAIS), Nankai University, Tianjin, China. His research interests include nonlinear control, visual servoing, AFM-based nano-systems, and control of underactuated systems.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on December 02,2025 at 07:17:45 UTC from IEEE Xplore. Restrictions apply.